

KSB System — Formalization

1. Pipeline and Stages

The pipeline consists of four stages in series:

Stage	Label	Length	Nominal exit velocity
Upstream	U	—	v_u
Conditioning	C	L^C	v_c
KSB (buffer)	B	L^B	v_d
Downstream	D	L^D	v_d

The **upstream** stage U is a pure source: it spawns inputs according to the stochastic pitch process and does nothing kinematic. It has no control length and runs at the nominal spawn velocity v_u implied by the pitch process. It is the first stage by convention so that everything kinematic happens *after* a clean stochastic source.

The **conditioning** stage C is where all upstream kinematic work happens. It runs at a nominal velocity $v_c > v_u$, which serves two purposes: it statically stretches the incoming pitch by the ratio $\eta_c = v_c/v_u$ (more spacing into the buffer — more time, more margin), and it carries the event-triggered feedforward controller that shapes velocity around skips. The detailed formalization is in *Conditioning formalization* (formerly *Upstream formalization*).

The **buffer** stage B exits directly at the downstream velocity v_d . The earlier KSB exit velocity v^{BR} and the registration stage have been removed: the buffer is mechanically tied to deliver inputs at v_d , with no intermediate registration belt.

Input length is l_i (uniform across batch, written l where unambiguous).

2. Stage Boundaries

A **stage boundary** $\mathcal{B}^k$ is the interface between two adjacent stages. It has a fixed absolute position and a required crossing velocity:

$$\mathcal{B}^k = (P^k, v^k)$$

Boundary	Position	Crossing velocity
$\mathcal{B}^{UC}$	$P^{UC} = 0$	$v^{UC} = v_u$
$\mathcal{B}^{CB}$	$P^{CB} = L^C$	$v^{CB} = v_c$
$\mathcal{B}^{BD}$	$P^{BD} = L^C + L^B$	$v^{BD} = v_d$

Every crossing velocity is a **design constant**. Setting it determines both the exit constraint of the stage before and the entry constraint of the stage after, simultaneously.

The source/conditioning boundary $\mathcal{B}^{UC}$ is where inputs leave the pure source and enter the conditioning belt. Because U is kinematically inert, an input crosses $\mathcal{B}^{UC}$ at the spawn velocity v_u . The conditioning belt then carries it to $\mathcal{B}^{CB}$, where it crosses into the buffer at v_c — the static stretch is realized over L^C as the faster belt spreads the spacing. The buffer delivers to $\mathcal{B}^{BD}$ at v_d .

3. The Straddling Constraint

When input i straddles boundary $\mathcal{B}^k$ — leading edge past P^k , trailing edge before it — both adjacent belts must run at v^k . This is a hard mechanical constraint imposed by the input's rigidity.

- **Leading edge arrives** at P^k at time $t_i^{k,\text{lead}}$. Both belts lock at v^k .
- **Straddling duration**: $\tau_i^k = l_i/v^k$.
- **Trailing edge clears** at $t_i^{k,\text{trail}} = t_i^{k,\text{lead}} + \tau_i^k$. The downstream stage gains full control authority at this moment.

During the straddling window:

$$v_i(t) = v^k \quad \forall, t \in [t_i^{k,\text{lead}}, t_i^{k,\text{trail}}].$$

The composite per-input trajectory is now **four segments**: conditioning, buffer, straddle, downstream. (The straddle term is what remains of the former `t_offset` after the registrar carry was removed.)

4. The Source and the Pitch Process

The upstream source U delivers a sequence of inputs to $\mathcal{B}^{UC}$. The arrival process is the **only** stochastic object in the pipeline; everything downstream is deterministic conditional on the arrival realization.

Parameters. Four parameters specify the source process:

Symbol	Meaning	Units
r_u	Mean upstream arrival rate	1/s
μ_u	Mean upstream pitch (center-to-center spacing of arrivals)	m
σ_u	Upstream pitch standard deviation	m
ρ_u	Pitch-process autocorrelation	—

The spawn velocity $v_u = r_u \mu_u$ is a **derived statistic of the source**, not an independent control parameter. This is the key change from the earlier model: velocity is no longer the controllable upstream quantity. The source owns the pitch statistics; the *conditioning* stage owns velocity. Decoupling them is what allows pitch into the buffer to be stretched (via v_c) without touching the stochastic pitch process itself.

Distributional model. Pitches are generated by an AR(1) process in log-space:

$$\log(p_i - p_{\min}) = \mu_y + \rho_u (\log(p_{i-1} - p_{\min}) - \mu_y) + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, \sigma_\varepsilon^2),$$

with μ_y and σ_ε chosen so that the marginal pitch p_i has mean μ_u and standard deviation σ_u , and with a hard floor $p_{\min}$ on pitch (typically $p_{\min} = l$) to forbid input overlap. The log-space construction enforces positivity without truncation, and the AR(1) structure captures clustering — a large pitch is more likely to be followed by another large pitch — which matters for any analysis that conditions on local arrival history.

5. Conditioning

The conditioning stage C converts the source's spawn velocity v_u into the buffer entry velocity v_c , and shapes the boundary conditions each input presents at $\mathcal{B}^{CB}$. It has two levers.

Static stretch. Running the conditioning belt at $v_c > v_u$ stretches the spacing of every input by the ratio

$$\eta_c = \frac{v_c}{v_u} \geq 1,$$

so the mean pitch entering the buffer is $\eta_c \mu_u$. More spacing means more free window and more kinematic margin at the early (entry-edge) segments — the entry-edge analogue of what the downstream rate ratio does for the exit edge. The stretch is static and global: it shifts the *mean* of the pitch-into-buffer distribution but not its shape. v_c is bounded above by the physical belt speed limit and, where it sets buffer entry velocity, by the buffer's own $v_{\max}$.

Reactive shaping. On top of the static baseline v_c , the conditioning controller adjusts velocity around events via a feedforward jerk timeline. The detailed treatment is in *Conditioning formalization*. The controller is open-loop and event-triggered: it computes corrections analytically from known events, not from a measured error. The existing skip-triggered policy (accelerate before a skip, decelerate after) and the planned anti-cluster policy (slow an input locally when two consecutive slots are filled, to add pitch reactively where co-occupancy pressure is highest) both live here. Where the static stretch widens spacing globally, the reactive policy widens it locally and on demand.

The conditioning belt's nominal velocity v_c , its speed ceiling, and the velocity gap to the source ($v_c - v_u$) are the design parameters of this stage.

6. Downstream Slots

The downstream side of the pipeline runs at constant belt velocity v_d and presents a fixed pattern of slots into which inputs must be delivered. The pattern is fully deterministic.

Symbol	Meaning	Units
r_d	Slot rate	1/s
μ_d (or d_s)	Slot pitch (center-to-center spacing of slots)	m
v_d	Slot pattern velocity, $v_d = r_d \mu_d$	m/s

Slot s passes $\mathcal{B}^{BD}$ at time s/r_d for $s = 0, 1, 2, \dots$, with the slot's position $p_s^{\text{slot}}(t)$ tracking the pattern at velocity v_d for all t .

The slot pattern specifies *which slots exist*. The assignment of inputs to slots — *which input goes to which slot* — is a separate scheduling decision, treated in §7.

7. Slot Assignment and the Skip Mechanism

Greedy slot assignment. Input i is assigned to the earliest slot index s_i that is both

(a) strictly later than the previous input's assignment, $s_i > s_{i-1}$, and (b) feasible under π^I 's kinematic budget given the input's entry state at $\mathcal{B}^{CB}$, the slot's arrival timing, and the buffer's available control window T_i^B .

Constraint (a) preserves order — inputs cannot swap slots — and constraint (b) is the kinematic feasibility test: can the buffer trajectory π^I deliver input i to slot s_i on time, given its entry

kinematics and the slot's downstream timing?

The policy is **greedy**: the earliest slot passing both constraints is always chosen, with no global lookahead and no reservation across inputs. Operationally, the slot picker bounds its search using π^I 's feasibility window (the range of time horizons T for which the solver can produce a trajectory) and walks forward through candidate slots $s_{i-1} + 1, s_{i-1} + 2, \dots$ until the first feasible one is found.

Skips. A skip is the event where input i is assigned to slot $s_i > s_{i-1} + 1$ — one or more slots between the two assignments go unfilled. Slot $s_{i-1} + 1$ was tried, found kinematically infeasible, and skipped; and so on, until the picker landed on a feasible slot. The **skip count** for input i is $s_i - s_{i-1} - 1 \geq 0$, zero in the typical case and one or more when slots go unfilled.

Skips are not actively scheduled. They are the emergent consequence of greedy assignment running up against the kinematic feasibility wall.

Load ratio. The structural rate of skipping is set by the load ratio:

$$\rho = \frac{r_u}{r_d}.$$

The pipeline operates with $\rho < 1$: arrivals are slower than slots, on average. In long-run averages, the fraction of slots filled is ρ and the fraction skipped is $1 - \rho$. The kinematic feasibility test determines *when* skips happen; the rate mismatch determines *how many* must happen. The mean interval between consecutive skip events, measured in inputs, is

$$Q = \frac{1}{1 - \rho}.$$

Both $1 - \rho$ and Q depend only on the mean rates and are invariant under changes in σ_u or ρ_u .

Phase error. The pressure that drives skips is captured by the phase error ϕ_i . Framed as a PLL: the upstream stream is the reference oscillator (r_u), the slot grid is the controlled oscillator (r_d), $\rho = r_u/r_d$, and a skip is a cycle slip. ϕ is defined reference-minus-controlled and references the *assigned* slot, not the nearest. The loop is open: the slot grid is free-running and fixed, with no loop filter. Both the buffer-side and upstream-side phase definitions (ϕ_b, ϕ_u) measure the same accumulated phase from opposite ends of the same transaction; under linear conditioning they are conserved ($\phi_u + \phi_b \approx \text{const}$).

Between skip events, ϕ_i drifts upward at rate $r_d - r_u$ per input, producing a characteristic sawtooth: a gradual upward ramp as the kinematic budget is eaten into, followed by a downward jump of approximately one slot period when a skip advances the picker to a later slot. ϕ is retained as an **analysis tool** — a measure of how hard the buffer is working — not as a load-bearing control object.

Individual skip intervals. The mean skip interval Q is fixed by ρ alone; individual intervals are not. Under low arrival variability ($\sigma_u \approx 0$), skips occur almost periodically. Under high variability, individual inputs arrive early or late, scattering individual intervals into a distribution around Q with spread growing in σ_u , while the long-run average stays exactly Q . Under AR(1) arrivals ($\rho_u > 0$), the distribution acquires clustering: runs of large pitches delay skips, then runs of small pitches trigger several in quick succession.

The two regimes have direct consequences downstream. In the low- σ_u regime the boundary conditions inherited by the buffer are predictable in timing and magnitude — edge pressure is clustered and forecastable. In the high- σ_u regime the same total pressure is dispersed across batch position.

8. Per-Input Control Windows

For each input i on stage S :

Symbol	Meaning
$t_i^{S,\text{ctrl-start}}$	Time trailing edge clears the entry boundary — control starts
$t_i^{S,\text{ctrl-end}}$	Time leading edge reaches the exit boundary — control complete
$T_i^S = t_i^{S,\text{ctrl-end}} - t_i^{S,\text{ctrl-start}}$	Control window duration
$\Lambda^S = L^S - l_i$	Effective control length

The solver for stage S on input i receives:

$$(\Lambda^S, v^{k_{\text{in}}}, v^{k_{\text{out}}}, T_i^S, \text{bounds})$$

where $v^{k_{\text{in}}}$ and $v^{k_{\text{out}}}$ are the crossing velocities of the entry and exit boundaries of stage S .

These are design constants — the solver never has to discover them. For the buffer, $v^{k_{\text{in}}} = v_c$ and $v^{k_{\text{out}}} = v_d$.

9. Entry State at Each Stage

The **entry state** $\mathbf{x}_i^S$ is the kinematic state of input i at $t_i^{S,\text{ctrl-start}}$.

$$\mathbf{x}_i^S = [0 \quad v^{k_{\text{in}}} \quad 0]^T.$$

- Position is zero (delta semantics — each stage uses its own local frame).

- Velocity is exactly $v^{k_{\text{in}}}$ by the straddling constraint.
- Acceleration is zero because the straddling window is at constant velocity.

Every solver therefore receives $a = 0$ at entry and must deliver $a = 0$ at exit — structurally guaranteed by the straddling constraint, not a convenience assumption.

10. The Policies

The pipeline is driven by named policies and one implied per-segment control. A **policy** is a function from boundary conditions, time horizon, and kinematic constraints to a commanded trajectory — not the trajectory itself, but the rule that generates one. The greedy slot assignment of §7 is a separate scheduling object that produces one of π^I 's inputs (the time horizon T_i^B); it is not a policy in this sense.

π^U — **Upstream (source) policy**. Null. The upstream stage spawns inputs and does nothing kinematic; it carries no policy. It is retained as a named stage only to keep the stochastic source cleanly separated from the conditioning kinematics.

π^C — **Conditioning policy**. Operates on the conditioning belt, between source and buffer. Detailed formalization in *Conditioning formalization*. π^C runs the belt at the nominal velocity v_c (the static stretch) and shapes velocity around events via a feedforward jerk timeline. It keeps the entry state input i presents at $\mathcal{B}^{CB}$ nominal — velocity v_c , acceleration zero — across event discontinuities. Open-loop and event-triggered. Holds both the existing skip-response policy and the planned anti-cluster policy. Kinematic bounds are the conditioning belt's hardware envelope.

π^I — **Per-input buffer trajectory**. Generates the trajectory each input follows from buffer entry to buffer exit. The input must traverse $\Lambda^B = L^B - l$ in time T_i^B (set by the slot picker per §7), entering at $(v_c, 0)$ and exiting at $(v_d, 0)$. The trajectory is jerk-limited under bounds set by the zero-slip assumption: while a segment bears an input, the input must not slide on the belt, which caps acceleration at the contact-friction limit. The jerk bound is set by servo capability under the no-slip regime. Detailed treatment in *Buffer formalization* §2.4.

π^S — **Per-segment synchronization**. Resolves segment-level synchronization within the free windows between consecutive inputs. When segment k has no input on it during the interval $W_{i,k} = t_{i+1,k}^{\text{in}} - t_{i,k}^{\text{out}}$, it must drive itself from the state delivered to input i at handoff to the state demanded by input $i + 1$ at receipt. Because no input is present during the free window, the zero-slip constraint does not bind: π^S operates under the servo's own kinematic envelope, strictly more permissive than π^I 's. The current implementation uses Ruckig's jerk-limited time-optimal trajectory under these bounds. Detailed treatment in *Buffer formalization* §2.

π_k^B — **Per-segment combined control (implied)**. What segment k physically does at every instant. When an input is on segment k , the segment is locked to that input's (v, a) by the zero-slip constraint, executing a slice of π^I ; when no input is on segment k , the segment executes π^S . Fully determined by π^I , π^S , and the schedule; not designed independently.

11. Notation Summary

Symbol	Type	Meaning
L^S	design	Length of stage S
v^k	design	Crossing velocity at boundary $\mathcal{B}^k$
v_u	derived	Spawn velocity of the source, $v_u = r_u \mu_u$
v_c	design	Conditioning belt nominal velocity; buffer entry velocity
η_c	design	Static stretch ratio $v_c/v_u \geq 1$
v_d	design	Downstream belt / slot velocity, $v_d = r_d \mu_d$
l_i	input	Physical length of input i
Λ^S	derived	Effective control length of stage S : $L^S - l_i$
τ_i^k	derived	Straddling duration at boundary k : l_i/v^k
$r_u, \mu_u, \sigma_u, \rho_u$	scenario	Source rate, mean pitch, pitch std dev, AR(1) autocorrelation
r_d, μ_d, v_d	derived	Downstream slot rate, slot pitch, slot velocity
ρ	derived	Load ratio r_u/r_d
Q	derived	Mean skip interval $1/(1 - \rho)$ in input units
ϕ_i	analysis	Phase error: reference-minus-controlled offset to assigned slot, units $1/r_d$
s_i	per-input	Assigned slot index for input i
$t_i^{S,\text{ctrl-start}}$	per-input	Time trailing edge clears entry boundary of stage S
$t_i^{S,\text{ctrl-end}}$	per-input	Time leading edge reaches exit boundary of stage S
T_i^S	per-input	Control window duration on stage S
$\mathbf{x}_i^S$	per-input	Entry state $[0, ; v^{k_{\text{in}}}, ; 0]^\top$
π^U	policy	Upstream (source) policy — null

Symbol	Type	Meaning
π^C	policy	Conditioning policy (static stretch + reactive shaping)
π^I, π^S	policy	Per-input buffer trajectory, per-segment synchronization
π_k^B	implied	Per-segment combined control on segment k